



PAPAYAYKWARE

Estudios científicos comprensibles

· INICIO

ARCHIVOS DEL BLOG



junio 21, 2026

DE UN MVP ROTO A UNA HERRAMIENTA USABLE: LO QUE APRENDÍ DEPURANDO OPEN-CAREER-COACH CON IA

Por **Javi Ciborro** (@papayaykware)

Hace unos días publiqué la primera versión de **OPEN-CAREER-COACH**, un proyecto open source pensado para algo muy concreto: ayudar a alguien a saber, en segundos, qué tan bien encaja su CV con una oferta de empleo concreta — y qué le falta para encajar mejor.

Lo que quiero contar aquí no es tanto el proyecto en sí, sino el proceso de pulirlo hasta dejarlo realmente usable. Porque entre "funciona en mi cabeza" y "funciona cuando alguien lo descarga y lo ejecuta" hay una distancia que solo se cierra con algo muy poco glamuroso: depuración sistemática. Y en este caso, esa depuración la hice de la mano de Claude, de Anthropic — no como una herramienta que escribe código por mí, sino como un colaborador técnico que audita, prueba y razona junto a mí.

Qué hace OPEN-CAREER-COACH (y qué no)

Antes de entrar en la parte interesante, lo esencial. El sistema hace tres cosas:

1. **Lee tu CV** (texto, PDF o DOCX) y extrae datos estructurados: habilidades técnicas, habilidades blandas, años de experiencia, contacto.
2. **Lee una oferta de empleo** y extrae sus requisitos de la misma forma.
3. **Calcula un score de compatibilidad**, combinando dos cosas: similitud semántica (usando *embeddings*, una técnica que convierte texto en vectores numéricos para medir cuán parecidos son dos textos en significado, no solo en palabras) y coincidencia directa de habilidades y experiencia.

No usa un modelo de lenguaje generativo todavía — no reescribe tu CV ni inventa una carta de presentación. Es, deliberadamente, una primera capa honesta: medir el encaje real antes de optimizar nada.

El momento incómodo: cuando el código "funciona" pero no funciona

Aquí está la parte que de verdad quiero compartir, porque creo que es donde está la lección.

El MVP estaba publicado. La estructura parecía sólida: módulos separados para CV, ofertas, matching, interfaz. Un README con badges verdes de "build passing". Todo parecía en orden a simple vista.

Pero cuando le pedí a Claude que revisara el repositorio en profundidad — no que lo describiera, sino que lo **ejecutara** — empezaron a aparecer grietas que ningún vistazo superficial habría detectado:

La interfaz no procesaba lo que el usuario pegaba. La app Gradio llamaba a un método (`process_text`) que sencillamente no existía en el código. El resultado: cualquiera que pegara su CV en el cuadro de texto se encontraba con un error críptico de archivo no encontrado, sin entender por qué.

El modo de línea de comandos ni siquiera arrancaba. Existían dos archivos con el mismo nombre — un módulo simple `cv_parser.py` y una carpeta `cv_parser/` con la lógica avanzada. Python, al encontrar ambos, siempre prioriza la carpeta. El archivo simple quedaba huérfano, invisible, inalcanzable. `python run_mvp.py` fallaba en la primera línea.

La app se rompía por un conflicto de versiones invisible. Una librería (`gradio`) dependía de una función que otra librería (`huggingface_hub`) había eliminado en una actualización reciente. El `requirements.txt` no fijaba versiones compatibles entre sí, así que dependiendo de cuándo instalaras el proyecto, podía simplemente no arrancar.

Y el propio README mentía, sin querer. El ejemplo de código de la documentación usaba una clase que nunca llegó a implementarse en el código real. Alguien que copiara ese ejemplo para entender cómo usar la herramienta se habría encontrado con un `ImportError` desde la primera línea.

Ninguno de estos errores era visible leyendo el código por encima. Todos eran visibles **ejecutándolo**.

Por qué esto importa más allá de este proyecto

Hay una tentación, cuando se trabaja con IA generativa, de pedir que "genere" algo y darlo por bueno porque tiene buena pinta: bien estructurado, bien comentado, con badges de estado. Pero el código no es prosa, su corrección no se mide en si se lee bien, sino en si **hace lo que dice que hace** cuando se ejecuta con datos reales, en condiciones reales, con las dependencias reales que alguien va a instalar.

Lo que distinguió este proceso no fue que Claude "escribiera" las correcciones, sino que las **verificó**, una por una, ejecutando el código real, antes de entregarlas. Cuando un fix parecía no funcionar por culpa de la red del propio entorno de pruebas y no del código, eso se distinguió explícitamente en vez de asumir que todo estaba resuelto. Cuando el README contradecía al código, se hizo una elección consciente: seguir siempre al código real, y no a la documentación aspiracional. Esa disciplina —dudar de lo que parece correcto hasta probarlo— es, creo, la diferencia entre una herramienta que "demuestra una idea" y una herramienta que alguien puede instalar hoy y usar para algo real.

El resultado: documentación que dice la verdad

Tras esta revisión, el repositorio quedó con:

- Una **guía de uso** que describe exactamente lo que el sistema hace hoy — no lo que hará en el roadmap — distinguiendo claramente el modo CLI simple del modo con interfaz gráfica y matching semántico completo.
- Un **README corregido**, con ejemplos de código que se pueden copiar y pegar sin que fallen.
- Un **CHANGELOG** con una entrada honesta sobre los errores encontrados y corregidos, en vez de ocultarlos o reescribir la historia.
- Y, sobre todo, un sistema que **arranca de verdad**, en ambos modos, en un entorno limpio.

Nada de esto es espectacular. No hay una nueva funcionalidad, ningún modelo nuevo, ningún salto de versión mayor. Es, simplemente, la diferencia entre un proyecto que *parece* terminado y uno que *está* terminado.

Por qué lo cuento aquí

Llevo tiempo sosteniendo, en este espacio y en el corpus que voy construyendo, que la colaboración entre humano e IA tiene más valor cuanto más tangible es. No se trata de delegar el pensamiento, sino de tener un colaborador capaz de hacer algo que a cualquiera, trabajando solo y bajo presión de tiempo, le cuesta: parar, dudar de lo evidente, y comprobarlo todo antes de darlo por bueno.

OPEN-CAREER-COACH es un proyecto pequeño. Pero si a una sola persona, en algún momento, le ayuda a entender mejor por qué su CV no está encajando con una oferta — o a identificar exactamente qué le falta para que sí encaje — entonces esta pequeña labor de depuración habrá tenido sentido. Y si quien lo usa quiere mirar bajo el capó, encontrará un código que hace lo que su documentación dice, lo cual, lamentablemente, no siempre se puede dar por hecho.

Resumen

- OPEN-CAREER-COACH es una herramienta open source que mide el encaje entre un CV y una oferta de empleo usando embeddings y coincidencia de habilidades.
- Tras su publicación inicial, se identificaron y corrigieron cuatro errores reales (interfaz que no procesaba texto, CLI que no arrancaba, conflicto de dependencias, documentación con ejemplos inexistentes en el código).

- La corrección se hizo verificando cada fix ejecutando el código real, no asumiendo que "parecía correcto".
- El repositorio, la guía de uso y el README están ahora alineados con lo que el código realmente hace.
- Repositorio: github.com/papayaykware/OPEN-CAREER-COACH

Conceptualización técnica del proceso de depuración y de este artículo: Claude (Anthropic).

Dirección del proyecto: Javi Caborro.

Compartir

COMENTARIOS



Escribe tu comentario

ENTRADAS POPULARES

febrero 19, 2025

GUÍA PRÁCTICA SOBRE TÉCNICAS DE XAI: SHAP, LIME Y GRAD-CAM

[Compartir](#) [Publicar un comentario](#)

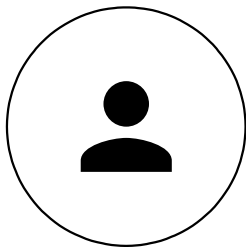
diciembre 17, 2024

N-ACETILCISTEÍNA (NAC): FUENTES ALIMENTARIAS NATURALES

[Compartir](#) [Publicar un comentario](#)

 Con la tecnología de Blogger

Puedes tener un doctorado y seguir siendo un idiota: Richard Feynman



Papayaykware

—
SANTA CRUZ DE
TENERIFE, SANTA
CRUZ DE TENERIFE,
Spain

[VISITAR PERFIL](#)

[Denunciar abuso](#)

Buscar
Buscar este blog

Buscar

Translate

Seleccionar idioma



Con la tecnología de  Traductor de Goo

